Ciclo Superior en Desarrollo de aplicaciones multiplataforma
Acceso a datos
UD. 1. Manejo de ficheros
Boletín de ejercicios

Ejercicios para trabajar con la clase File

1) Crear una aplicación llamada *MakeDirectories* que se admita las siguientes posibles ejecuciones:

a. MakeDirectories path1 path2 …

Creará los directorios indicados por parámetro. Serán tantos como se indique.

b. MakeDirectories -d path1 path2 …

Borrará los directorios indicados en los parámetros.

c. MakeDirectories -r path1 path2

Renombra el archivo indicado como *path1* con el nombre *path2*.
Después de renombrar un archivo, se indicarán los siguientes datos acerca de él:

- Ruta absoluta.
- Si es de solo lectura.
- Se admite modificación.
- Nombre.
- Ruta.
- Tamaño.
- Fecha de su última modificación.

Si el usuario no hace uso correcto de la aplicación se mostrará un mensaje por pantalla indicando cómo debe usarse. Por ejemplo:

```
Uso: MakeDirectories path1 ....
   Crea cada path
Uso: MakeDirectories -d path1 ...
   Borra cada path
Uso: MakeDirectories -r path1 path2
   Renombra path1 a path2
```

Nota: la aplicación recibe los datos por parámetro. En Java se almacenan en el vector de String del método *main(String[] args)*. Por lo tanto, no es preciso realizar ninguna lectura de información desde la entrada estándar.

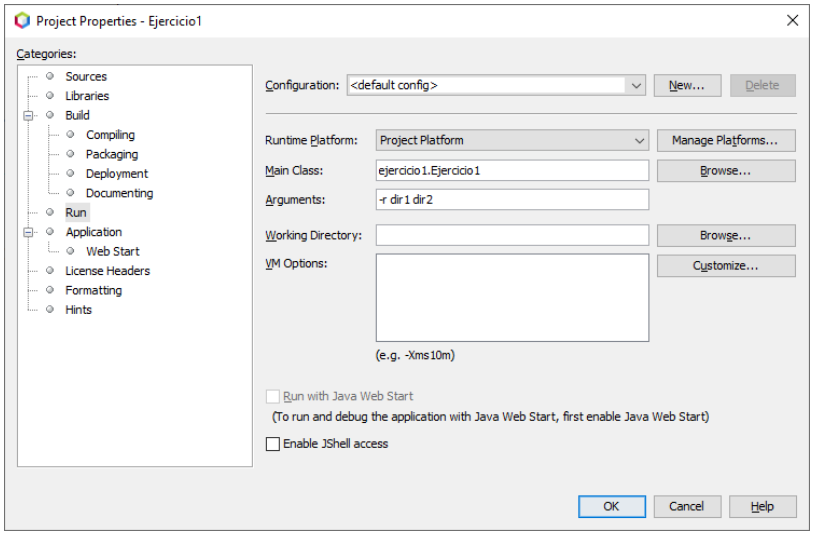
Para probar la aplicación se pueden utilizar distintas técnicas que se comentan a continuación.
Ejecutar la aplicación desde línea de comandos

Independientemente de que se utilice el intérprete de comandos de Windows, una ventana de PowerShell de Windows o un terminal en Linux, el comando a ejecutar para probar la aplicación será, por ejemplo:

```
Java MakeDirectories -r dir1 dir2
```

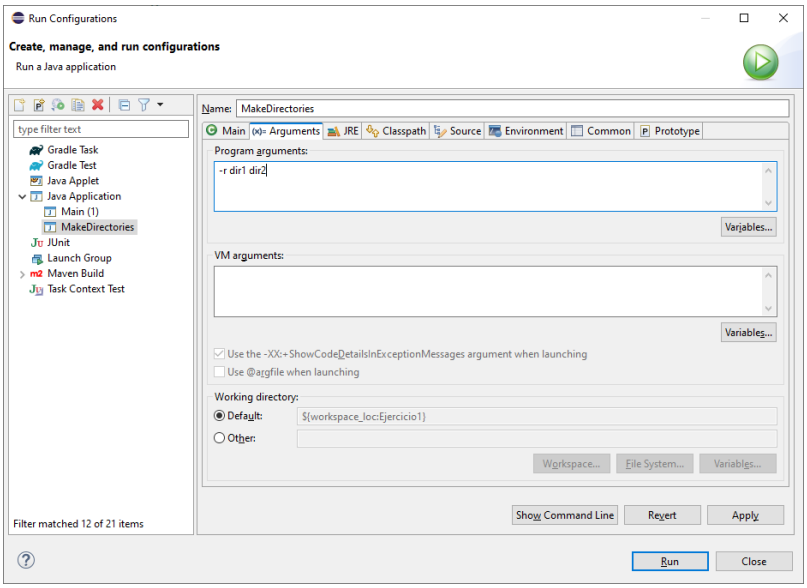
Desde NetBeans

En NetBeans se pueden indicar los parámetros en las propiedades del proyecto:

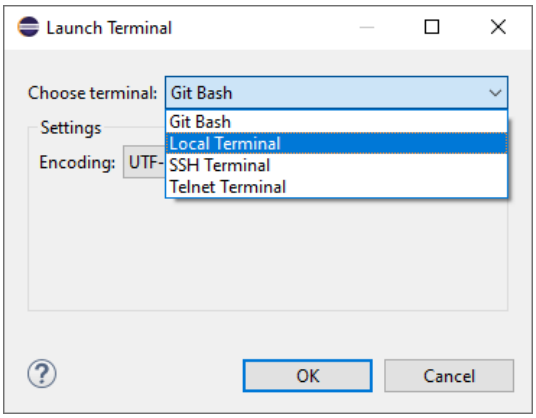


Eclipse

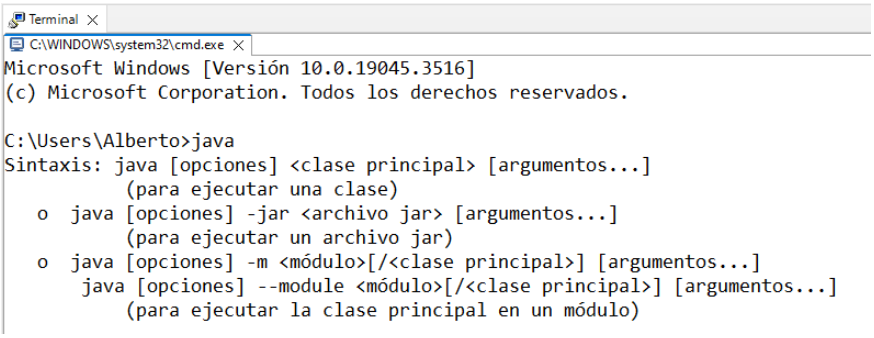
Eclipse también permite configurar el entorno para que, al ejecutar la aplicación, se le envíen los parámetros que se desee. Se puede hacer en la opción *Run configurations…* del menú de ejecución:



Alternativamente, Eclipse admite la opción de ejecutar un terminal desde el propio IDE. Se debe seleccionar la opción *Local Terminal*:



Desde el terminal se puede ejecutar la aplicación llamando directamente a la aplicación Java seguida del nombre de la aplicación y sus parámetros en caso de que los hubiese:



2) Escribe una clase *Utilidades* con los siguientes métodos:

a. crearLista(ruta_directorio, ruta_fichero): crea un fichero de texto que contenga los nombres de las entradas de un directorio.
b. copiarFicheros(origen1, origen2, destino): debe unir los contenidos de dos ficheros en uno solo. Los ficheros no tienen por qué ser de texto.
c. moverFichero(ruta_fichero, nueva_ruta): mueve un fichero desde una ubicación a otra.

Codifica llamadas de prueba a estos métodos para probar la funcionalidad de la clase Utilidades.

3) Escribir una utilidad que admita cero o más expresiones regulares que representen directorios.

La utilidad deberá mostrar los nombres de los directorios y de los archivos que hay en ellos y que cumplan con el patrón.

Se recomienda crear una clase que contenga archivos y directorios y que implemente la interfaz Iterable para poder recorrer sus elementos.

Cuando la clase tenga que procesar un directorio, añadirá cada elemento a la colección de archivos o directorios según sea su tipo. Este procesado deberá ser recursivo para procesar el contenido de los directorios.

Un ejemplo de salida donde se procesa la ruta actual donde reside un proyecto de tipo NetBeans, sería la siguiente:

```
Dirs: [
   .\build
   .\build\classes
   .\build\classes\util
   .\build\empty
   .\build\generated-sources
   .\build\generated-sources\ap-source-output
   .\dist
   .\nbproject
   .\nbproject\private
   .\src
   .\src\util
   .\test
]

Files: [
   .\build\built-jar.properties
   .\build\classes\.netbeans_automatic_build
   .\build\classes\.netbeans_update_resources
   .\build\classes\util\Directory$1.class
   .\build\classes\util\Directory$TreeInfo.class
   .\build\classes\util\Directory.class
   .\build\classes\util\Print.class
   .\build.xml
   .\dist\README.TXT
   .\dist\utilidadesDirectorio.jar
   .\manifest.mf
   .\nbproject\build-impl.xml
   .\nbproject\genfiles.properties
   .\nbproject\private\private.properties
   .\nbproject\private\private.xml
   .\nbproject\project.properties
   .\nbproject\project.xml
   .\src\util\Directory.java
   .\src\util\Print.java
]
```

4) El siguiente código lista los archivos que contiene un directorio. Admite por parámetro una expresión regular para filtrar aquellos contenidos que sean de interés:

```
package ejercicio3;

import java.io.File;
import java.io.FilenameFilter;
import java.util.Arrays;
import java.util.regex.Pattern;

class DirFilter implements FilenameFilter {

    private final Pattern pattern;

    public DirFilter(String regex){
        pattern = Pattern.compile(regex);
    }

    @Override
    public boolean accept(File dir, String name) {
        return pattern.matcher(name).matches();
    }
}

public class Ejercicio3v0 {

    public static void main(String[] args) {
        File path = new File(".");
        String[] list;

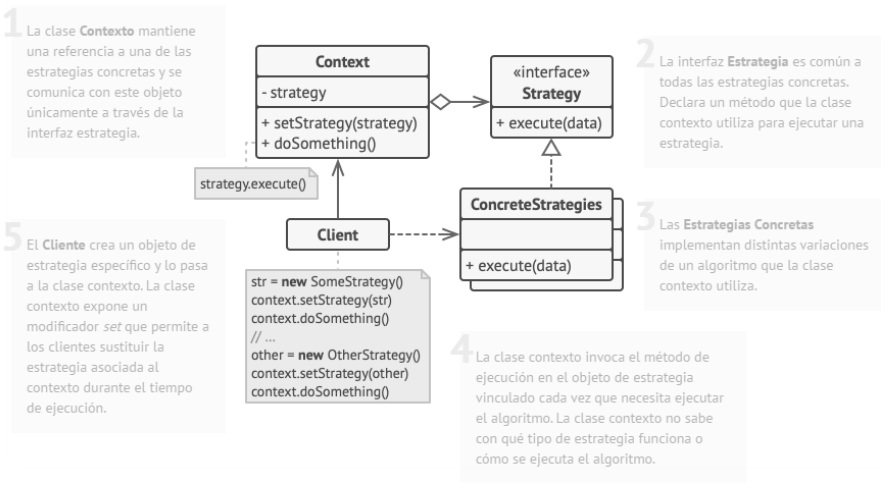
        if (args.length == 0)
            list = path.list();
        else
            list = path.list(new DirFilter(args[0]));

        Arrays.sort(list);

        for (String dirITem: list)
            System.out.println(dirITem);
    }
}
```

Obsérvese que la única razón de existir *DirFilter* es proporcionar el método *accept()* al método *list()*, de modo que éste pueda determinar qué nombres de archivos deben incluirse en la lista.

Es un ejemplo del patrón de diseño *Strategy* porque *list()* implementa la funcionalidad básica y proporciona la estrategia en la forma de un objeto.



Se pide:

a. Reescribir el código anterior de tal modo que se sustituya la clase *DirFilter* por una clase interna anónima. Al implementar esta clase, la llamada a *path.list*() se sustituirá por:

```
list = path.list(new DirFilter(args[0]);  => list.path.list(filter(args[0]);
```

Obsérvese que se sustituye la creación de una clase por la llamada a un método. La clase *DirFilter*, proporcionada en el listado anterior, implementa la interfaz *FilenameFilter*.

El método *filter()* ahora debe devolver una instancia de una clase que implemente la interfaz *FilenameFilter* y que haga el mismo trabajo que la anterior.

b. Modificar la clase *Dirlist* para que el objeto *FilenameFilter* abra y lea cada archivo y lo acepte si alguno de los argumentos de la línea de comandos existe en el fichero.

c. Crear una clase llamada *SortedDirList* con un constructor que admita un objeto de tipo *File* y construya una lista ordenada con los ficheros contenidos en *File*. Codificar dos métodos sobrecargados *list()* de tal modo que uno retorne una lista completa y otro un subconjunto que corresponda con su argumento que será una expresión regular.

d. Modificar la clase *DirList* para que calcule la suma de los tamaños de los archivos seleccionados.

5) Escribir un programa que cuente el número de líneas de un fichero de texto. La aplicación mostrará cada línea precedida por la numeración que le corresponde.

6) Escribir un programa que reciba por parámetro una o más rutas de ficheros de texto. Si no se recibe ninguno, mostrará un mensaje de error y finalizará.

La aplicación escribirá el nombre de cada fichero junto con su número de líneas.

Si ocurriese un error al procesar un archivo, la aplicación mostrará un mensaje de error para el archivo, pero seguirá procesando el resto.

7) Escribir una aplicación que reciba por parámetro un fichero de texto y lo lea línea a línea. Debe realizar las siguientes tareas:

a. Almacenar las líneas en un contenedor *LinkedList*. Imprimir todas las líneas extrayéndolas del contenedor.
b. Admitir un parámetro opcional (-M) que, si está presente como segundo argumento, convierta las líneas leídas a mayúsculas.
c. Admitir tantos parámetros como especifique el usuario con palabras. Mostrar solo las líneas que contienen alguna de estas palabras.

No es preciso comprobar posibles errores en la entrada, pero sí es conveniente hacer una codificación que admita fácilmente incorporar nuevas opciones.

8) Cuando en Java se escribe una instrucción como la siguiente:

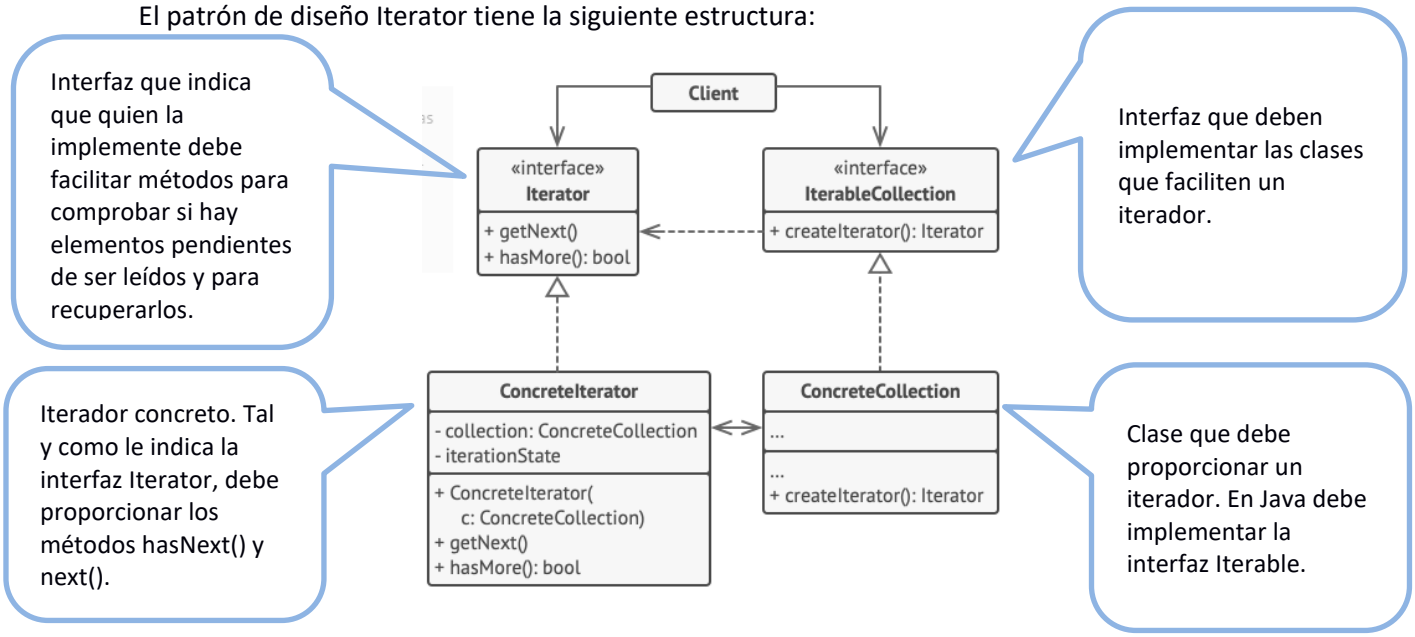
```
BufferedReader br = new BufferedReader(new inputStreamReader(new InputStream(…)));
```

se está haciendo uso del patrón de diseño *Decorator*. El objeto *InputStream* está siendo envuelto por un *InputStreamReader* y este objeto, de nuevo, está envuelto por el *BufferedReader*. Cada una de estas clases agrega funcionalidad tal y como muestran los apuntes del tema. Por ejemplo, *InputStreamReader* añade la capacidad de leer caracteres o, en el caso del *BufferedReader*, añade la capacidad de leer líneas completas de modo eficiente.

Se pide crear nuestra propia clase *BufferedReader* de tal modo que sea capaz de recibir en su constructor cualquier instancia de la clase abstracta Reader como, por ejemplo, un *InputStreamReader* o un *FileReader*. La clase debe poder utilizarse en una estructura *try-with-resources*.

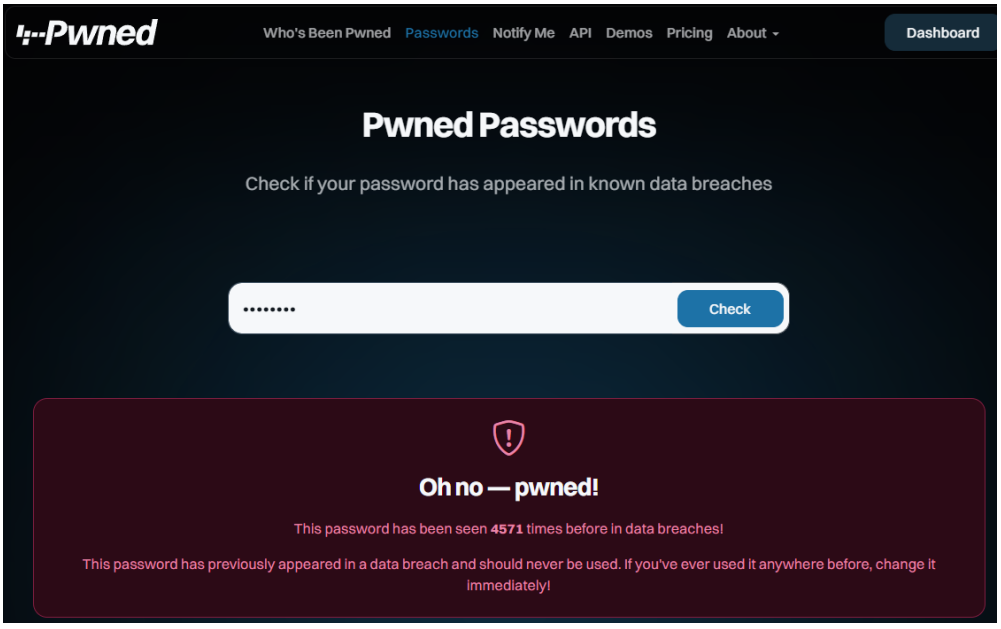
9) Escribir una clase que represente un fichero de texto y añadirle un iterador que permita ir recorriendo sus líneas. Implementar los métodos *hasNext()* para que indique si quedan elementos pendientes de procesar y el método *next()* para que devuelva el siguiente elemento.

El patrón de diseño Iterator tiene la siguiente estructura:



10) La página web https://haveibeenpwned.com/ fue creada por Troy Hunt, director regional de Microsoft y MVP como desarrollador orientado a la seguridad. El propósito de la página es ofrecer información sobre brechas de seguridad en correos y contraseñas.

Para este ejercicio nos centraremos en las contraseñas. La página ofrece una entrada donde el usuario puede escribir una contraseña y la web indica si figura en alguna base de datos que circula entre la comunidad de hackers. Por ejemplo, en la siguiente captura, se teclea la contraseña *abc123.,* y la web indica que ya está registrada:



El objetivo de la práctica es el siguiente: dado que nos puede surgir la duda razonable de qué ocurre con la contraseña que tecleamos (¿queda un registro de ella para usarse en futuros ataques?) se propone utilizar la API que proporciona la propia página web. Está accesible en https://haveibeenpwned.com/API/v3.

En el apartado Pwned paswords – Searching by a range, se propone lo siguiente:

- El usuario genera el código SHA1 en su ordenador de la contraseña.

  SHA-1 (Secure Hash Algorithm 1) es un algoritmo que transforma cualquier dato (texto, fichero, contraseña, …) en una cadena de 160 bits, normalmente representada como 40 caracteres hexadecimales. Hoy en día no se considera seguro.

  No es un algoritmo de cifrado. Estos transforman información utilizando una clave que después se puede descifrar. Las funciones hash, como SHA-1, transforma datos en una huella de tamaño fijo.

- Se llama a la URL https://api.pwnedpasswords.com/range/ añadiendo al final los cinco primeros caracteres del SHA1 generado en el paso anterior. De este modo, es imposible que un potencial atacante reconstruya la contraseña.

- La llamada a la URL anterior devuelve varias líneas con el siguiente formato:

  0018A45C4D1DEF81644B54AB7F969B88D65:1
  00D4F6E8FA6EECAD2A3AA415EEC418D38EC:2
  011053FD0102E94D6AE2F8B83D76FAF94F6:1
  012A7CA357541F0AC487871FEEC1891C49C:2
  0136E006E24E7D152139815FB0FC6A50B15:2
  ...

  La web devuelve los siguientes 35 caracteres del SHA1 de contraseñas registradas cuyos cinco primeros caracteres del SHA1 sean coincidentes con los que enviamos. Después de los dos puntos aparece un número que indica cuántas veces aparece la contraseña.

- Para saber si nuestra contraseña está registrada, hay que revisar manualmente la lista anterior para ver si hay coincidencias.

Se pide hacer un programa que lea contraseñas de un fichero de texto que recibirá la aplicación por parámetro. Para cada contraseña, comprobará si está registrada utilizando la técnica descrita anteriormente.

Generación del SHA1

```
MessageDigest digest = MessageDigest.getInstance("SHA-1");
digest.reset();
digest.update(contraseña.getBytes("utf8"));
sha1 = String.format("%040x", new BigInteger(1,
                    digest.digest())).toUpperCase();
// %040x: El cero anterior a 40x rellenar con ceros no significativos
```

Llamada a la API y lectura de la información

El siguiente código es más comúnmente aceptado hasta el JDK 11:

```
URL web = new URL(url);
HttpURLConnection con = (HttpURLConnection) web.openConnection();
BufferedReader br = new BufferedReader(new InputStreamReader(con.getInputStream()));
```

A partir del JDK 11 (incluido) es más aconsejable el uso de las siguientes clases:

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder()
                      .uri(URI.create(WEB))
                      .build();

HttpResponse<Stream<String>> response =
       client.send(request, HttpResponse.BodyHandlers.ofLines());
List<String> lista = response.body().toList();

// lista no admite modificaciones. Si son necesarias se puede utilizar la
// siguiente instrucción:
// List<String> lista = new ArrayList<>(response.body().toList());
```

Ampliación

Generalizar el código para que admita un fichero de contraseñas y las compruebe una a una.

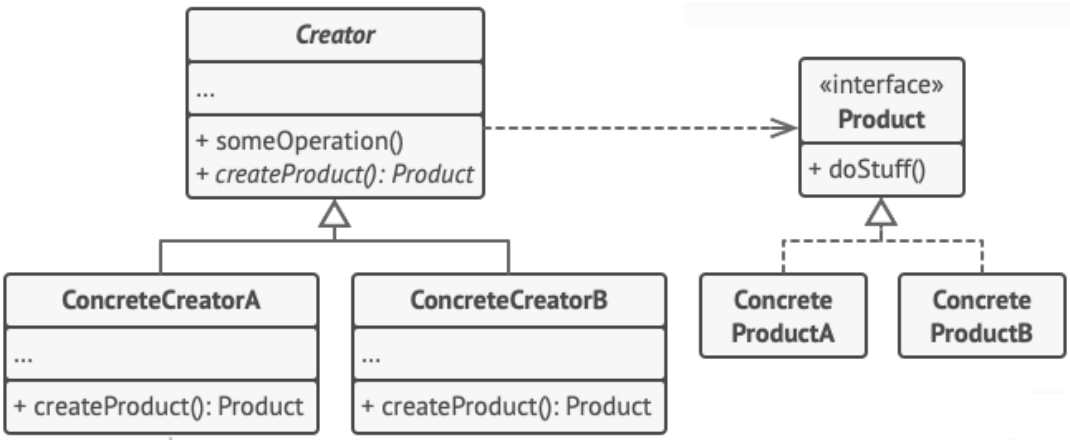
11) Desarrolla un sistema de importación de clientes desde ficheros de texto. Deben leerse los datos de los clientes desde los ficheros de texto y llevarlos a objetos Cliente en memoria.

Los ficheros de texto presentan un cliente por línea conteniendo su nombre, ciudad, email y fecha de nacimiento, pero su estructura varía ya que estos datos pueden estar separados por distintos elementos que pueden ser los siguientes:

- Coma (,): CSV
- Tabulación (\t): TSV
- Punto y coma (;): archivo de texto europeo

Requerimientos

El código deberá resolver el problema implementando el patrón de diseño *Factory Method*:



La aplicación debe:

- Detectar automáticamente el tipo de archivo según el separador que esté usando.
- Crear la clase que corresponda para leer el fichero dependiendo de qué tipo de archivo se trate.
- Esta clase ofrecerá la funcionalidad necesaria para obtener una estructura con instancias de la clase Cliente conteniendo los datos leídos.

Consideraciones adicionales

- Lectura de ficheros: Los ficheros de log estarán en formato de texto plano, y deben leerse línea por línea.
- Estructura del proyecto:
    - DataImporter (interfaz)
        - Ofrecerá el método *List<Cliente> importData(File file)*
    - Clases concretas para procesar los archivos:
        - CSVImporter
        - TSVImporter
        - SemicolonImporter
    - Fábrica:
        - DataImporterFactory con el método *DataImporter getImporter(File file)* que reciba el fichero y devuelva una clase que sea capaz de procesarlo.
    - Main (punto de entrada de la aplicación)

Ayuda

Para detectar el tipo de separador se puede utilizar el código que considere el alumno. El siguiente código puede servir de orientación:

```
public static char detectSeparator(File file) throws IOException {

    try (BufferedReader br = new BufferedReader(new FileReader(file))) {
        String firstLine = br.readLine();
        if (firstLine == null) throw new IOException("Archivo vacío");

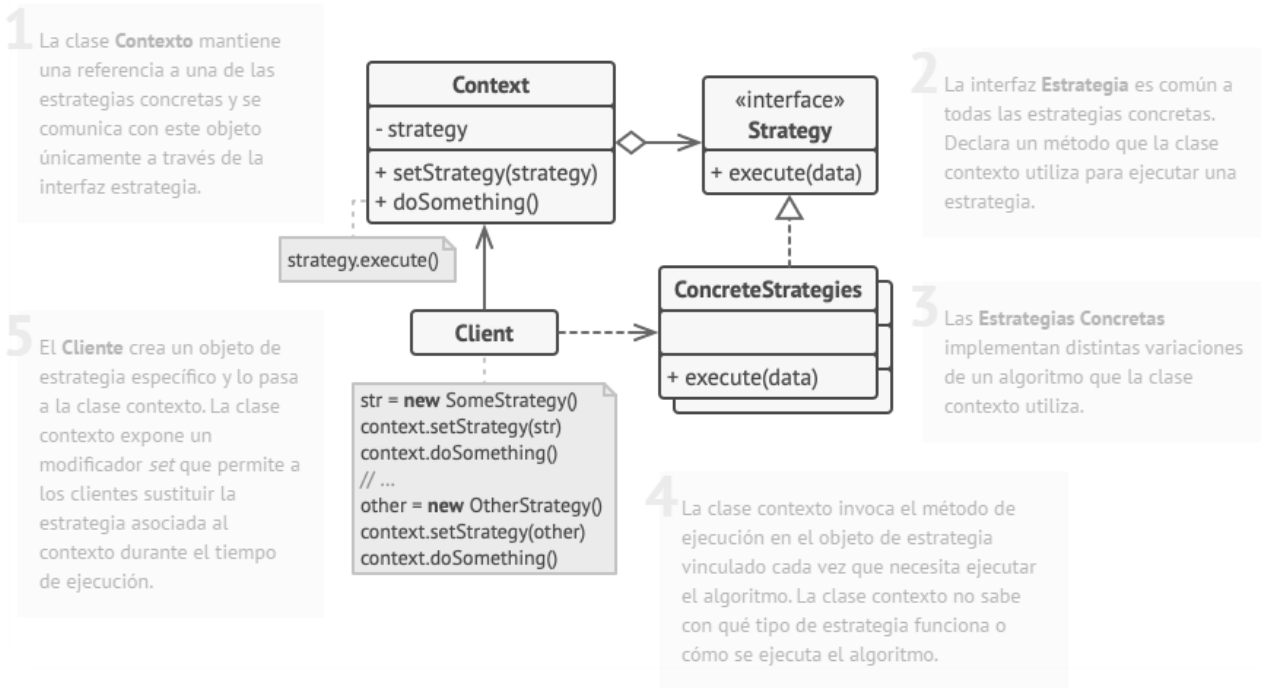
        int commas = firstLine.split(",").length;
        int tabs = firstLine.split("\t").length;
        int semicolons = firstLine.split(";").length;

        if (tabs > commas && tabs > semicolons)
               return '\t';
        else if (semicolons > commas)
               return ';';
        else
               return ','; // CSV por defecto
    }
}
```

12) Utilizando el fichero de texto plano *Cotizaciones.txt* que se facilita, escribe una clase Cotizaciones que realice las siguientes funciones:

a. leer(): lectura del fichero y volcado de los datos a una colección de objetos *Cotizacion* que tendrán nombre de la empresa, hora de la cotización, cotización, variación y capitalización.
b. getList(): devolver un *List* con todas las cotizaciones.
c. getMapa(): devolver un *Map* con todas las cotizaciones.
d. Codificación de métodos para recuperar la siguiente información:
    i. Empresa con mayor subida.
    ii. Empresa con mayor bajada.
    iii. Empresa con mayor capitalización.
    iv. Recodificar los métodos anteriores para que acepten un valor numérico que representen cuántas empresas se quieren recuperar. Por ejemplo, si al primer método se le pasa el valor *n*, devolverá las *n* empresas con mayor subida.
e. getVariacion(boolean): devuelve las empresas que tengan una variación positiva si el método recibe un valor positivo, variación negativa en caso contrario.
f. double getCotizacion(String): a partir del nombre de una empresa, devuelve su cotización.
g. getCotizacionesOrdenadas(): devuelve un *List* con las cotizaciones ordenadas por variación de mayor a menor.
h. grabar(): graba en un fichero los nombres de empresa y cotización ordenados por nombre de empresa.

13) Trabajando con el fichero de texto plano *cotizaciones.txt*, escribir una aplicación que permita aplicar distintas estrategias de análisis sobre los datos sin modificar el código principal cada vez que se quiera cambiar el tipo de análisis. Para lograrlo se aplicará el patrón *Strategy*.



Ayuda para la codificación

Se creará la interfaz *AnalisisStrategy*:

```
public interface AnalisisStrategy {
    void analizar(List<Empresa> empresas);
}
```

y se implementarán varias estrategias de análisis concretas:

- mayorVariacion(List<Empresa> empresas, int n): muestra las n empresas con mayor variación positiva.
- mayorCapitalizacion(List<Empresa> empresas, int n): muestra las n empresas con mayor capitalización.
- mayorRango(List<Empresa> empresas, int n): muestra las n empresas con mayor diferencia entre el valor máximo y el valor mínimo cotizado.

Por ejemplo, un posible método que analizase la mayor subida podría ser el siguiente:

```
public class mayorSubida implements AnalisisStrategy {
    @Override
    public void analizar(List<Empresa> empresas) {
        Empresa max =
          Collections.max(empresas, Comparator.comparing(Empresa::getVariacion));
        System.out.println("Empresa con mayor subida: " + max);
    }
}
```

Si se quisiesen ver las n mayores subidas, se podría implementar del siguiente modo:

```
public static List<Empresa> mayorVariacion(List<Empresa> empresas, int n) {
    return empresas.stream()
       // de mayor a menor

       .sorted(Comparator.comparing(Empresa::getVariacion).reversed())
       .limit(n) // nos quedamos con las n primeras
       .collect(Collectors.toList());
}
```

Para localizar, por ejemplo, los mayores rangos entre valores más altos y bajos cotizados, se puede programar del siguiente modo:

```
public static List<Empresa> mayorRango(List<Empresa> empresas, int n) {
     return empresas.stream()
            .sorted(Comparator
            .comparing(e -> e.getMaximo() - e.getMinimo())
            .reversed())
            .limit(n)
            .collect(Collectors.toList());
}
```

Se creará una clase *AnalizadorIbex* que implemente el patrón *Strategy*:

```
public class AnalizadorIbex {
    private AnalisisStrategy estrategia;
```

```
    public void setEstrategia(AnalisisStrategy estrategia) {
        this.estrategia = estrategia;
    }

    public void ejecutarAnalisis(List<Empresa> empresas) {
        estrategia.analizar(empresas);
    }
}
```

Finalmente, en el método *main* hay que hacer las siguientes tareas:

- Leer el fichero de texto.
- Preguntar al usuario qué tipo de análisis quiere hacer.
- Configurar la estrategia que corresponda y ejecutarla.

14) Se facilita el fichero plano *canciones.txt* que contiene una lista de canciones, una en cada línea. De cada canción se dispone de un título, nombre del grupo, duración en minutos y estilo.

Lectura de las canciones

Programar una clase *GestorCanciones* con un método que permita leer las canciones y otro que permita recuperarlas en un objeto tipo *List<Cancion>*.

Eliminación de duplicados

Programar un método en *GestorCanciones* que, a partir de las canciones registradas, elimine las canciones duplicadas. Para ello debe aplicarse la siguiente estrategia:

- La clase *Cancion* debe implementar una sobreescritura del método *equals(Object obj)*. Se hace del siguiente modo:
  - Primero comprobará si el objeto enviado tiene la misma referencia que la canción que implementa el método *equals*.
  - Si el objeto recibido no es una canción se responderá falso.
  - Se tratará el objeto recibido como una instancia de *Cancion*. El método *equals(Object)* recibe un *Object* porque es una sobreescritura del método implementado en la clase base.
  - Se responderá que se trata de la misma canción si el título y grupo coinciden.

- Posteriormente las canciones se almacenarán en un conjunto y Java precisa poder obtener un código único por canción para asignarle un espacio dentro de la colección. Este código lo facilita el método *hashCode()*. Se puede implementar como

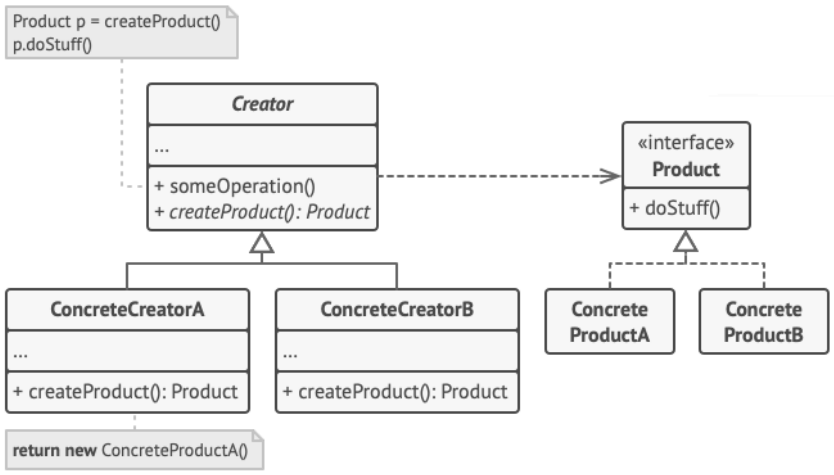
```
titulo.hashCode() * 31 + grupo.hashCode()
```

  Se multiplica por 31 porque es un número primo y genera una distribución razonablemente buena de hashes.

- Para eliminar los elementos repetidos, se pueden leer todas las canciones y añadirlas a una estructura de tipo conjunto. Por el hecho de ser un conjunto, los elementos repetidos se eliminarán. Si el conjunto es de tipo *HashSet<Cancion>*, no se garantiza que se respete el orden de las canciones. Para garantizarlo se puede utilizar un objeto de tipo *LinkedHashSet<Cancion>*.

Creación de PlayList

El usuario podrá crear tres tipos distintos de playlist a partir de las canciones registras, cada una con sus propias reglas. La aplicación delegará en una clase *Factory* la creación de la clase que gestiona cada una de las playlists por lo que usará el patrón de diseño *Factory Method*:



La clase GestorCancion ofrecerá el método *Playlist crearPlayList(String tipo)* que, a partir del texto, "ROCK", "POP" o "MIX", devolverá una clase especialista en cada tipo de playlist. Este método es el *método factory*.

Una posible ejecución de la aplicación podría ser la siguiente:

```
GestorCanciones gestor = new GestorCanciones("canciones.txt");

Playlist playlist = gestor.crearPlaylist("ROCK");

playlist.guardar("playlist.txt");
```

El valor devuelto por el método *factory* es un interfaz y debe tener dos métodos:

```
void generar();
void guardar(String nombreFichero) throws IOException;
```

Cada playlist realiza las siguientes tareas:

PlaylistRock

Objetivo: crear una playlist exclusivamente de rock.

Reglas:

- Solo puede incluir canciones cuyo estilo sea Rock.
- La duración máxima es de 60 minutos.
- Las canciones se recorren en el orden en el que aparecen en el fichero.
- Si una canción hace que se supere el límite de 60 minutos, no se añade, pero se continúa examinando el resto. Si se alcanzan los 60 minutos, se detiene el proceso.
- No puede haber canciones repetidas.

PlaylistPop

Reglas:

- Solo admite canciones Pop.
- Duración máxima 45 minutos.
- Como máximo puede contener 15 canciones.
- Se intenta conseguir una playlist lo más larga posible sin superar los 45 minutos.
- No puede haber canciones repetidas.

PlaylistMix

Reglas:

- Puede contener canciones de cualquier estilo.
- Duración máxima 60 minutos.
- No puede haber dos canciones consecutivas del mismo estilo.
- No puede haber canciones repetidas.

Guardar la playlist

Las tres clases utilizan el mismo formato de salida. Esta salida puede tener un aspecto como el que se muestra a continuación:

```
PLAYLIST ROCK
==============================

1. Bohemian Rhapsody - Queen
   Duración: 354 segundos
   Estilo: ROCK

2. Stairway to Heaven - Led Zeppelin
   Duración: 482 segundos
   Estilo: ROCK

3. Hotel California - Eagles
   Duración: 391 segundos
   Estilo: ROCK

==============================
Número de canciones: 3
Duración total: 1227 segundos
```